

# claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-aaa0059cc0cb52477.jsonl`  
- SHA-256: `0c9f9a8449c4d3bbff120a3cd7426d285039469af3b5a7c57dfbc1959291e982`  
- Source modified: `2026-06-22T01:30:51+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_2be29ecf-a31`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

## Transcript

### 1. user (2026-06-22T01:26:03.400Z)

You are reviewing PR #310 "P2b-install" for CONSISTENCY with the locked plan + REGRESSION risk. Run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and read docs/PACKAGING\_PLAN.md changes + backend/main.py + the new tests (tests/test\_p2b\_install.py, tests/test\_app\_launcher.py) in C:\Users\avidu\Projects\bhi-p2b. Scrutinize: (1) Does the change honor the locked L-decisions (LIGHT torch-free, no native component, default-OFF for --server/CLI)? (2) Is O8 resolution (truly-local + motion) recorded honestly in §7/§8/§9 + changelog, matching the code? (3) Do the new tests actually assert the IMPORTANT behavior (no-overwrite idempotency, effective motion config, the --app wiring) or are they shallow/tautological? (4) Any test that could be flaky (the subprocess --app seeding test ? port, timing, RALLY\_APP\_HOME isolation)? (5) Did anything break --server / CLI byte-identity? Report concrete findings + fixes. Empty list if nothing real.

### 2. assistant (2026-06-22T01:26:07.512Z)

I'll review PR #310. Let me start by examining the diff and the relevant files.

### 3. user (2026-06-22T01:26:10.669Z)

78e423d docs(pkg): link P2b-install PR #310 in the §7 tracker  
a5c1493 feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run default (gate O8)

### 4. assistant (2026-06-22T01:26:13.006Z)

Let me look at the full diff and the key files.

### 5. user (2026-06-22T01:26:13.977Z)

```
README.md | 31 ++++++  
backend/config/__init__.py | 9 +-  
backend/config/loader.py | 34 ++++++  
backend/main.py | 15 ++++  
docs/PACKAGING_PLAN.md | 13 +--  
scripts/install.py | 210 ++++++  
scripts/uninstall.py | 150 ++++++  
tests/test_app_launcher.py | 42 ++++++  
tests/test_p2b_install.py | 163 ++++++  
9 files changed, 660 insertions(+), 7 deletions(-)
```

### 6. user (2026-06-22T01:26:17.304Z)

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py  
index 9b276c0..6c38999 100644
```

```

--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -6,7 +6,12 @@ Public API:
 This package is the canonical place for all configuration access.
 """

-from .loader import load_config, save_default_config
+from .loader import (
+    LIGHT_DEFAULT_CONFIG,
+    load_config,
+    save_default_config,
+    write_light_default_config,
+)
+from .models import ( # re-export common ones
+    AppConfig,
+    Config,
@@ -27,6 +32,8 @@ from .startup import (
__all__ = [
    "load_config",
    "save_default_config",
+    "write_light_default_config",
+    "LIGHT_DEFAULT_CONFIG",
    "AppConfig",
    "Config",
    "DeploymentConfig",
diff --git a/backend/config/loader.py b/backend/config/loader.py
index 37ed101..274c4f7 100644
--- a/backend/config/loader.py
+++ b/backend/config/loader.py
@@ -212,6 +212,40 @@ def save_default_config(path: str | Path | None = None) -> Path:
     return cfg_path

+# The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
+# Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies skip_ai_handoff,
+# local storage and the compute_target, so this stays the handful of knobs a fresh, non-technical
+# user gets ? a torch-free, offline, zero-setup experience:
+# * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
+# * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-bills on Gemini nor returns 0 rallies on WASB-without-a-key (08 intent);
+# * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it either way).
+# The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
+LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
+    "sport": "badminton",
+    "deployment": {"profile": "truly-local"},
+    "indexing": {"default_segmenter": "motion", "use_gpu": False},
+}
+
+def write_light_default_config(
+    path: str | Path | None = None, *, overwrite: bool = False
+) -> tuple[Path, bool]:
+    """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
+
+    Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL AppConfig
+    and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by default ?
+    **never**
+    overwrites an existing file**, so a user's edits or the first-run wizard's choices are preserved
+    (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent dirs.
```

```

+ """
+ cfg_path = Path(path) if path is not None else default_config_path()
+ if cfg_path.exists() and not overwrite:
+     return cfg_path, False
+ cfg_path.parent.mkdir(parents=True, exist_ok=True)
+ cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n", encoding="utf-8")
+ return cfg_path, True
+
+
+ # Backwards-compatible alias used by some older call sites during transition.
+ def get_default_config() -> dict[str, Any]:
+     """Return defaults as a plain dict (for transitional use only)."""
diff --git a/backend/main.py b/backend/main.py
index c1738b4..4e5a19a 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -48,6 +48,7 @@ from backend.utils.single_instance import acquire_lock # P0e: one instance
per
from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
from backend.config import ( # new typed config
    load_config as load_app_config,
+ write_light_default_config, # P2b: batteries-included first-run config seeding
    AppConfig,
    StitchingConfig,
    enforce_startup_checks,
@@ -934,6 +935,20 @@ def main():
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

+ # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
+ # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
+ # no API key, no GPU, no cloud bills) at the resolved app-home (default_config_path(),
which
+ # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE load_app_config
reads
+ # it ? so a fresh `pip`/`uv` tool install` + `indexer --app` just works. NEVER overwrites an
+ # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to
--app
+ # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
+ if args.app:
+     seeded_path, seeded = write_light_default_config()
+     if seeded:
+         print(f"[APP] First run: seeded a truly-local default config at {seeded_path} "
+             f"(detector 'motion' ? offline, no API key or GPU needed). "
+             f"Use the in-app setup to switch to Gemini.")
+
+ # Initialize Global Config and DB
+ global global_config, db_instance
+ global_config = load_app_config() # returns typed AppConfig

```

## 7. assistant (2026-06-22T01:26:24.938Z)

Now let me look at how `args.app` is wired and confirm `--server` / CLI paths are unchanged. Let me see the surrounding context in main.py.

## 8. user (2026-06-22T01:26:26.827Z)

```

132:# --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? 01)
---
153:     """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
867:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")

```

```

868:     parser.add_argument("--app", action="store_true",
938:     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly `--app`
942:     # it ? so a fresh `pip`/`uv` tool install + `indexer --app` just works. NEVER overwrites
an
943:     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped to
--app
944:     # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
945:     if args.app:
996:     # CLI processing mode (no web UI needed). --app/--server always run the server, never
CLI single-shot.
997:     if args.video_path and not args.server and not args.app:
1078:     if args.app or args.server or not args.video_path:
1079:         if args.app:
1080:             # P2: the friendly launcher ? always localhost-only (use --server for network
exposure),
1089:             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).

```

## 9. user (2026-06-22T01:26:29.795Z)

```

130
131
132     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
133
134     def _port_is_free(host: str, port: int) -> bool:
135         """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
136         uvicorn surfaces a real bind failure loudly)."""
137         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
138             try:
139                 s.bind((host, port))
140                 return True
141             except OSError:
142                 return False
143
144
145     def _find_free_port(host: str) -> int:
146         """An OS-assigned free port on host."""
147         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
148             s.bind((host, 0))
149             return int(s.getsockname()[1])
150
151
152     def _resolve_app_port(host: str, port: int) -> int:
153         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
154         return port if _port_is_free(host, port) else _find_free_port(host)
155
156
157     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
158         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
159         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
160         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
161         if os.environ.get("RALLY_NO_BROWSER"):
162             return False
163         import time as _time
164         import urllib.request
165         deadline = _time.monotonic() + timeout
166         while _time.monotonic() < deadline:
167             try:

```

```

168         with urllib.request.urlopen(health_url, timeout=2) as r:
169             if getattr(r, "status", 200) == 200:
170                 break
171         except Exception:
172             _time.sleep(0.3)
173     else:
174         return False
175     try:
176         webbrowser.open(url)
177     except Exception:
178         pass
179     return True
180
181
182     # Serves static frontend files directly (now under api/ per approved structure).
183     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
184     # an arbitrary directory finds the shipped assets instead of creating a stray
185     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
186     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
187     try:
188         os.makedirs(_STATIC_DIR, exist_ok=True)
189     except OSError:
190         # The dir ships with the package (a no-op create); guard against a read-only install
191         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
192         # PermissionError.
193         pass
194     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
195
196     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
197     # snapshot
198     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
199     # if it's absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
200     # no SPA change is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
201     # is enough
202     # (no client-side path routing ? no history-fallback catch-all, which would also be a
203     # route-order hazard).
204     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
205     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
206     else _STATIC_DIR
207     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
208     if os.path.isdir(_UI_ASSETS_DIR):
209         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
210
211
212     @app.get("/", response_class=HTMLResponse)
213     def serve_index():
214         index_path = os.path.join(_UI_DIR, "index.html")
215         if os.path.exists(index_path):
216             with open(index_path, "r", encoding="utf-8") as f:
217                 return f.read()
218         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
219
220
221     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
222         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
223         bundled.
224         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
225         guard)."""
226         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
227         try:
228             with open(path, "r", encoding="utf-8") as f:

```

```

222         return json.load(f)
223     except (OSError, ValueError):
224         return None
225
226     # Global variables initialized at startup
227     db_instance: Optional[Database] = None
228     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
229     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
230     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
231     _instance_lock: Optional[Any] = None
232
233     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
234     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
235     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
236     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
237     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
238     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
239     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
240     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
241         JobWaiting,
242         _arm_wake_timer,
243         _drain_due_jobs,
244         _get_executor,
245         _job_to_request,
246         _MAX_DRAIN_WAKE_SEC,
247         _maybe_record_job_cost,
248         _run_claimed_job,
249         _schedule_next_drain_if_waiting,
250     )
251
252     # Legacy thin wrapper for any remaining direct calls during transition.
253     # New code should import load_app_config / AppConfig from backend.config directly.
254     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
255         cfg = load_app_config(config_path)
256         return cfg.model_dump(mode="json")
257
258     # --- API Models ---
259     class ProcessRequest(BaseModel):
260         video_path: str
261         player_pool: Optional[List[str]] = None
262         max_frames: Optional[int] = None
263         segmenter: Optional[str] = None
264         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
265         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
266         locale: Optional[str] = None
267
268     class QueryRequest(BaseModel):
269         video_id: str
270         server: Optional[str] = None
271         receiver: Optional[str] = None
272         duration_min: Optional[float] = None
273         event_type: Optional[str] = None
274
275     class CompileRequest(BaseModel):
276         video_id: str
277         segment_ids: List[int]
278         output_filename: str
279         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel

```

```

watermark
280     # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
281     locale: Optional[str] = None
282
283     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
284         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
285
286         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
287         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
288         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
289         """
290         if segments:
291             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
292         else:
293             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
294
295
296     # --- API Endpoints ---
297     @app.get("/api/config")
298     def get_config():
299         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
300         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
301         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
302         # by tests/test_redact.py + the /api/config redaction test).
303         if global_config is None:
304             return {}
305         return redact_secrets(global_config.model_dump(mode="json"))
306
307     @app.get("/api/health")
308     def health():
309         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
310         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
311         confirm the engine is reachable and which detector is wired. Never raises."""
312         sport = getattr(global_config, "sport", None)
313         indexing = getattr(global_config, "indexing", None)
314         return {
315             "status": "ok",
316             "sport": sport,
317             "default_segmenter": getattr(indexing, "default_segmenter", None),
318         }
319
320
321     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
322     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
323     API_VERSION = 1
324
325
326     def _engine_version() -> str:
327         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
328         try:
329             return _pkg_version("badminton-highlight-indexer")

```

## 10. assistant (2026-06-22T01:26:32.805Z)

Now let me look at the `truly-local` profile preset to verify it actually supplies  
`skip\_ai\_handoff`, local storage, and compute\_target as the config comment claims.

## 11. user (2026-06-22T01:26:34.639Z)

```
backend/config/loader.py:216:# Deliberately MINIMAL: the `truly-local` PROFILE preset
(presets.py) supplies skip_ai_handoff,
backend/config/loader.py:220:# * profile "truly-local" ? no Gemini handoff, local
storage ? never surprise-bills on
backend/config/loader.py:226:     "deployment": {"profile": "truly-local"},
backend/config/models.py:362:     profile: Literal["", "truly-local", "cloud-serving", "cpu-
mock"] = ""
backend/config/presets.py:4:config knobs, so the truly-local / cloud-serving / cpu-mock modes
don't drift apart one
backend/config/presets.py:27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-
mock")
backend/config/presets.py:33:     "truly-local": "local_gpu",
backend/config/presets.py:44:     "truly-local": {
backend/config/presets.py:45:         "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend/config/presets.py:118:     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
backend/config/presets.py:131:         return "truly-local"
backend/config/startup.py:3:A deployment profile (`truly-local` / `cloud-serving` / `cpu-
mock`) implies a coherent
backend/config/startup.py:187:     if profile == "truly-local":
backend/config/startup.py:188:         # truly-local reads bytes from the local FS or a *mounted*
Drive ? a cloud backend here is
backend/config/startup.py:193:         f"profile 'truly-local' usually uses local/gdrive
storage, but storage.backend="
Binary file backend/config/__pycache__/loader.cpython-313.pyc matches
Binary file backend/config/__pycache__/models.cpython-313.pyc matches
Binary file backend/config/__pycache__/presets.cpython-313.pyc matches
Binary file backend/config/__pycache__/startup.cpython-313.pyc matches
```

## 12. user (2026-06-22T01:26:38.266Z)

```
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
```



```

53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()

```

```

107
108
109 def suggest_profile(
110     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111 ) -> str:
112     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114     Signals, in priority order:
115
116         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117         - ``CI`` truthy                             ? ``"cpu-mock"``
118         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119         - otherwise                                 ? ``""`` (no strong suggestion)
120
121     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
122     never
123     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
124     passes
125     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
126     on it.
127
128     """
129     e = os.environ if env is None else env
130     if e.get("K_SERVICE"):
131         return "cloud-serving"
132     if _truthy(e.get("CI")):
133         return "cpu-mock"
134     if probe_cuda and _cuda_available():
135         return "truly-local"
136     return ""

```

### 13. assistant (2026-06-22T01:26:42.236Z)

The preset confirms the comment: ``truly-local` supplies skip_ai_handoff: True, storage.backend: local, and compute_target: local_gpu. Now I need to verify that the preset is actually APPLIED when loading config ? i.e., that writing {"deployment": {"profile": "truly-local"}} actually expands to skip_ai_handoff: True. Let me check the loader.`

### 14. user (2026-06-22T01:26:43.390Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (
23         COMPUTE_TARGET_FOR_PROFILE,
24         PROFILE_PRESETS,
25         is_known_profile,

```

```

26         preset_for,
27         suggest_profile,
28     )
29
30     logger = logging.getLogger(__name__)
31
32     # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33     # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34     # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35     # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36     # from a later RALLY_APP_HOME.)
37
38
39     def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40         """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
41         replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42         overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43         path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44         out = dict(base)
45         for key, val in override.items():
46             if isinstance(val, dict) and isinstance(out.get(key), dict):
47                 out[key] = _deep_merge(out[key], val)
48             else:
49                 out[key] = val
50         return out
51
52
53     def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54         """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55         is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56         environment never silently selects a profile (and never silently spends on cloud).
57
58         An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59         profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60         asserting a preset that was never applied. A bad value in config.json is left to
surface as
61         a hard, typed-Literal error downstream (config-file correctness)."""
62         env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63         if env_profile and env_profile.strip():
64             ep = env_profile.strip()
65             if is_known_profile(ep):
66                 return ep
67             logger.warning(
68                 "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69                 "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70             )
71             # fall through to the file's profile (env typo must not suppress a valid file
choice)
72         dep = file_data.get("deployment")
73         if isinstance(dep, dict):
74             prof = dep.get("profile")
75             if isinstance(prof, str) and prof.strip():
76                 return prof.strip()

```

```

77         return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
81     "") -> list[str]:
82         """Dotted paths in `data` that the typed config will not honor.
83
84         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
85         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
86         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
87         user's intent is lost ? collect every such key so load_config can warn.
88         """
89         unknown: list[str] = []
90         fields = model_cls.model_fields
91         for key, val in data.items():
92             if key not in fields:
93                 unknown.append(prefix + key)
94                 continue
95             ann = fields[key].annotation
96             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
97             BaseModel):
98                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
99
100         return unknown
101
102
103     def _get_builtin_defaults() -> dict[str, Any]:
104         """Return a plain dict of the current built-in defaults.
105
106         The AppConfig model is the single source of truth for defaults.
107         """
108         return AppConfig().model_dump(mode="json")
109
110
111     def load_config(path: str | Path | None = None) -> AppConfig:
112         """Load configuration with the following precedence:
113
114         1. Built-in defaults defined in the Pydantic models.
115         2. Values from the JSON file (if present and readable).
116         3. (Future) environment / CLI overrides.
117
118         Missing file or unreadable file ? returns a fully defaulted AppConfig.
119         Unknown keys in the file are tolerated (extra="allow" on the model).
120         """
121         cfg_path = Path(path) if path is not None else default_config_path()
122
123         file_data: dict[str, Any] = {}

```

#### 15. user (2026-06-22T01:26:46.651Z)

```

120         file_data: dict[str, Any] = {}
121         if cfg_path.exists():
122             try:
123                 with open(cfg_path, "r", encoding="utf-8") as f:
124                     file_data = json.load(f)
125             except Exception as exc:
126                 # Non-fatal: continue with defaults + whatever we could parse.
127                 # In production we might log here.
128                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
129                 data.")
130
131             # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
132             `null`) would
133             # otherwise crash startup: a truthy non-mapping hits `.items()` in
134             _unknown_key_paths, and any
135             # non-mapping breaks the `**defaults, **file_data` unpack. Degrade to defaults +

```

```

warn, per the
133     # loader's "bad input is non-fatal" contract.
134     if not isinstance(file_data, dict):
135         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
136                        cfg_path, type(file_data).__name__)
137     file_data = {}
138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(
143                 "[CONFIG WARNING] %s contains keys the typed config does not "
144                 "recognize (typo'd or removed knobs ? top-level extras are kept "
145                 "but unread; unknown section keys are silently dropped): %s",
146                 cfg_path, ", ".join(sorted(unknown)),
147             )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180             elif ct and canonical and ct != canonical:
181                 logger.warning(
182                     "[CONFIG] deployment profile %r normally uses compute_target=%r but
"

```

```

183             "config.json overrides it to %r ? honoring the explicit override.",
184             profile, canonical, ct,
185         )
186         logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                     profile, merged["deployment"].get("compute_target"))
188     else:
189         # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190         merged = {**defaults, **file_data}
191         suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192         if suggestion:
193             logger.debug(
194                 "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
195                 %r ? "
196                 "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
197                 "auto-applies ? D15).", suggestion,
198             )
199         return AppConfig.model_validate(merged)
200
201
202 def save_default_config(path: str | Path | None = None) -> Path:
203     """Write a fresh default config.json.
204
205     Used by scripts/doctor.py diagnostics and the --setup CLI path.
206     Overwrites if the file exists.
207     """
208     cfg_path = Path(path) if path is not None else default_config_path()
209     cfg = AppConfig()
210     cfg_path.parent.mkdir(parents=True, exist_ok=True)
211     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212     return cfg_path
213
214

```

## 16. assistant (2026-06-22T01:26:50.759Z)

The loader confirms the profile preset is applied at load time. The minimal ``LIGHT_DEFAULT_CONFIG`` will be expanded to include ``skip_ai_handoff: True``, ``storage.backend: local``, and ``compute_target: local_gpu``. Now let me read the tests.

## 17. user (2026-06-22T01:26:51.279Z)

```

1     """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3     Covers:
4     * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5     the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader
produces;
6     * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7     installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/I0.
8     """
9     from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16

```

```

17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
exec)."""
22         path = Path(__file__).resolve().parents[1] / "scripts" / name
23         spec = importlib.util.spec_from_file_location(name[:-3], path)
24         assert spec and spec.loader
25         mod = importlib.util.module_from_spec(spec)
26         spec.loader.exec_module(mod)
27         return mod
28
29
30     # --- engine seeding: write_light_default_config (gate 08)
-----
31
32     def test_seed_writes_truly_local_motion(tmp_path):
33         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
34         path, written = write_light_default_config(cfg)
35         assert written is True and path == cfg and cfg.exists()
36         data = json.loads(cfg.read_text(encoding="utf-8"))
37         assert data == LIGHT_DEFAULT_CONFIG
38         assert data["indexing"]["default_segmenter"] == "motion"
39         assert data["indexing"]["use_gpu"] is False
40         assert data["deployment"]["profile"] == "truly-local"
41
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
encoding="utf-8")
46         path, written = write_light_default_config(cfg)
47         assert written is False and path == cfg
48         # The user's file is preserved untouched (the wizard/edits are never clobbered).
49         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
== "gemini"
50
51
52     def test_seed_overwrite_true_forces(tmp_path):
53         cfg = tmp_path / "config.json"
54         cfg.write_text("{}", encoding="utf-8")
55         _, written = write_light_default_config(cfg, overwrite=True)
56         assert written is True
57         assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
58
59
60     def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61         """The minimal seeded file, run through the loader, expands via the truly-local
PRESET to a
62         coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
GPU)."""
63         cfg = tmp_path / "config.json"
64         write_light_default_config(cfg)
65         loaded = load_config(cfg)
66         assert loaded.indexing.default_segmenter == "motion"
67         assert loaded.indexing.use_gpu is False
68         assert loaded.deployment.profile == "truly-local"
69         assert loaded.indexing.skip_ai_handoff is True # supplied by the truly-local
preset
70         assert loaded.storage.backend == "local" # supplied by the truly-local
preset
71
72
73     # --- scripts/install.py pure logic

```

```

-----
74
75     @pytest.fixture(scope="module")
76     def install_mod():
77         return _load_script("install.py")
78
79
80     @pytest.fixture(scope="module")
81     def uninstall_mod():
82         return _load_script("uninstall.py")
83
84
85     @pytest.mark.parametrize(
86         "system, fname, executable",
87         [("Windows", "KhelSutra.bat", False), ("Darwin", "KhelSutra.command", True),
88          ("Linux", "khelsutra.sh", True)],
89     )
90     def test_shortcut_spec_per_os(install_mod, system, fname, executable):
91         home = Path("/home/u/AppHome")
92         name, content, ex = install_mod.shortcut_spec(system, home)
93         assert name == fname and ex is executable
94         # str(home) is OS-native (backslashes on Windows) ? assert against it, not a
95         hardcoded literal.
96         assert "RALLY_APP_HOME" in content and str(home) in content
97         assert "indexer --app" in content
98
99     def test_resolve_method(install_mod):
100         assert install_mod.resolve_method("auto", uv_present=True) == "uv"
101         assert install_mod.resolve_method("auto", uv_present=False) == "pip"
102         assert install_mod.resolve_method("pip", uv_present=True) == "pip" # explicit wins
103         assert install_mod.resolve_method("uv", uv_present=False) == "uv"
104
105
106     def test_install_command_argv(install_mod):
107         assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
108         pip = install_mod.install_command("pip", "badminton-highlight-indexer")
109         assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111
112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
114         r"C:\Users\u\AppData\Roaming"})
115         assert win.name == "Khelsutra" and "Roaming" in str(win)
116         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
117         "/home/u/.local/share"})
118         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
119
120
121     def test_build_manifest_shape(install_mod):
122         home = Path("/home/u/AppHome")
123         m = install_mod.build_manifest(
124             method="uv", app_home=home, shortcut=home / "khelsutra.sh",
125             created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
126             timestamp="2026-06-22T00:00:00+00:00",
127         )
128         assert m["package"] == "badminton-highlight-indexer"
129         assert m["install_method"] == "uv"
130         assert str(home / "config.json") in m["user_data"] # user data is tracked
131         separately (kept on uninstall)
132         assert m["shortcut"] == str(home / "khelsutra.sh")
133         assert m["schema"] == 1
134
135     # --- scripts/uninstall.py pure logic

```



```

-----
133
134 def test_uninstall_command_argv(uninstall_mod):
135     assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
136         ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
137     pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
138     assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
139
140
141 def test_find_manifest_explicit(uninstall_mod, tmp_path):
142     m = tmp_path / "uninstall_manifest.json"
143     m.write_text("{} ", encoding="utf-8")
144     assert uninstall_mod.find_manifest(str(m)) == m
145     assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
146
147
148 def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
149     """The manifest install.py would write is consumable by uninstall.py's
150     expectations."""
151     home = tmp_path / "AppHome"
152     m = install_mod.build_manifest(
153         method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
154         created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
155         timestamp="2026-06-22T00:00:00+00:00",
156     )
157     mp = home / "uninstall_manifest.json"
158     home.mkdir(parents=True)
159     mp.write_text(json.dumps(m), encoding="utf-8")
160     assert uninstall_mod.find_manifest(str(mp)) == mp
161     loaded = json.loads(mp.read_text(encoding="utf-8"))
162     assert uninstall_mod.uninstall_command(loaded["install_method"],
163         loaded["package"])[0:3] in (
164         ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
165     )
166

```

## 18. assistant (2026-06-22T01:26:53.651Z)

Now the app launcher test, which is the one most likely to be flaky.

## 19. user (2026-06-22T01:26:54.321Z)

```

1     """The friendly `indexer --app` launcher (PACKAGING_PLAN.md P2)."""
2
3     import os
4     import socket
5     import subprocess
6     import sys
7     import time
8     import urllib.error
9     import urllib.request
10
11     import pytest
12
13     import backend.main as m
14
15     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
16
17
18     def _free_port() -> int:
19         s = socket.socket()
20         s.bind(("127.0.0.1", 0))
21         p = int(s.getsockname()[1])
22         s.close()
23         return p

```

```

24
25
26 # ---- port helpers ----
27
28 def test_find_and_check_free_port():
29     p = m._find_free_port("127.0.0.1")
30     assert isinstance(p, int) and p > 0
31     assert m._port_is_free("127.0.0.1", p) is True
32
33
34 def test_resolve_app_port_keeps_free_port():
35     p = _free_port()
36     assert m._resolve_app_port("127.0.0.1", p) == p
37
38
39 def test_resolve_app_port_falls_back_when_busy():
40     s = socket.socket()
41     s.bind(("127.0.0.1", 0))
42     s.listen()
43     busy = int(s.getsockname()[1])
44     try:
45         assert m._port_is_free("127.0.0.1", busy) is False
46         alt = m._resolve_app_port("127.0.0.1", busy)
47         assert alt != busy
48         assert m._port_is_free("127.0.0.1", alt) is True
49     finally:
50         s.close()
51
52
53 # ---- open-browser-when-ready ----
54
55 class _FakeResp:
56     status = 200
57     def __enter__(self): return self
58     def __exit__(self, *a): return False
59
60
61 def test_open_browser_opens_once_healthy(monkeypatch):
62     monkeypatch.delenv("RALLY_NO_BROWSER", raising=False)
63     monkeypatch.setattr(urllib.request, "urlopen", lambda *a, **k: _FakeResp())
64     opened = {}
65     monkeypatch.setattr(m.webbrowser, "open", lambda u: opened.setdefault("url", u) or
66 True)
67     assert m._open_browser_when_ready("http://127.0.0.1:1/",
68 "http://127.0.0.1:1/api/health", timeout=2) is True
69     assert opened["url"] == "http://127.0.0.1:1/"
70
71
72 def test_open_browser_suppressed_by_env(monkeypatch):
73     monkeypatch.setenv("RALLY_NO_BROWSER", "1")
74     monkeypatch.setattr(m.webbrowser, "open", lambda u: pytest.fail("must not open when
75 suppressed"))
76     assert m._open_browser_when_ready("http://x/", "http://x/api/health") is False
77
78
79 def test_open_browser_gives_up_when_never_ready(monkeypatch):
80     monkeypatch.delenv("RALLY_NO_BROWSER", raising=False)
81
82     def _boom(*a, **k):
83         raise OSError("connection refused")
84
85     monkeypatch.setattr(urllib.request, "urlopen", _boom)
86     monkeypatch.setattr(m.webbrowser, "open", lambda u: pytest.fail("must not open a
87 dead page"))
88     assert m._open_browser_when_ready("http://x/", "http://x/api/health", timeout=0.4)

```

```

is False
85
86
87     # ---- end-to-end: `indexer --app` boots the server + serves the bundled UI ----
88
89     def test_app_mode_boots_and_serves_bundled_ui(tmp_path):
90         """`indexer --app` (browser suppressed) must start the server localhost-only and
serve both
91         /api/health and the bundled SPA at /."""
92         port = _free_port()
93         env = {
94             **os.environ,
95             "RALLY_NO_BROWSER": "1", # don't pop a browser in CI
96             "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
97         }
98         log = open(tmp_path / "app.log", "wb")
99         proc = subprocess.Popen(
100             [sys.executable, "-m", "backend.main", "--app", "--port", str(port),
101              "--output-dir", str(tmp_path / "out")],
102             cwd=REPO_ROOT, env=env, stdout=log, stderr=subprocess.STDOUT,
103         )
104         base = f"http://127.0.0.1:{port}"
105         try:
106             deadline = time.time() + 90
107             healthy = False
108             while time.time() < deadline:
109                 if proc.poll() is not None:
110                     pytest.fail(f"--app server exited early (code {proc.returncode}); see
{tmp_path/'app.log'}")
111                 try:
112                     with urllib.request.urlopen(f"{base}/api/health", timeout=2) as r:
113                         if r.status == 200:
114                             healthy = True
115                             break
116                     except (urllib.error.URLError, ConnectionError, OSError):
117                         time.sleep(0.5)
118                 assert healthy, "the --app server never became healthy"
119                 with urllib.request.urlopen(f"{base}/", timeout=5) as r:
120                     body = r.read().decode("utf-8", "replace")
121                 assert "/assets/" in body # the bundled SPA index.html (single-origin asset
refs)
122             finally:
123                 proc.terminate()
124                 try:
125                     proc.wait(timeout=10)
126                 except subprocess.TimeoutExpired:
127                     proc.kill()
128                 log.close()
129
130
131     def test_app_mode_seeds_truly_local_config(tmp_path):
132         """First `indexer --app` run with an EMPTY app-home seeds the truly-local 'motion'
default
133         config (PACKAGING_PLAN.md P2b / gate 08) BEFORE serving ? proving the main() wiring
fires."""
134         import json
135         port = _free_port()
136         app_home = tmp_path / "apphome"
137         app_home.mkdir()
138         env = {
139             **os.environ,
140             "RALLY_NO_BROWSER": "1",
141             "RALLY_APP_HOME": str(app_home), # isolate: never touch the real OS app-data
dir
142             "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),

```

```

143     }
144     log = open(tmp_path / "app.log", "wb")
145     proc = subprocess.Popen(
146         [sys.executable, "-m", "backend.main", "--app", "--port", str(port)],
147         cwd=tmp_path, env=env, stdout=log, stderr=subprocess.STDOUT,
148     )
149     cfg = app_home / "config.json"
150     try:
151         deadline = time.time() + 90
152         seeded = False
153         while time.time() < deadline:
154             if cfg.exists(): # check the file FIRST: seeding precedes (and is
independent of) serving
155                 seeded = True
156                 break
157             if proc.poll() is not None:
158                 pytest.fail(f"--app exited before seeding (code {proc.returncode}); see
{tmp_path/'app.log'}")
159                 time.sleep(0.4)
160                 assert seeded, "config.json was not seeded under RALLY_APP_HOME"
161                 data = json.loads(cfg.read_text(encoding="utf-8"))
162                 assert data["indexing"]["default_segmenter"] == "motion"
163                 assert data["deployment"]["profile"] == "truly-local"
164         finally:
165             proc.terminate()
166             try:
167                 proc.wait(timeout=10)
168             except subprocess.TimeoutExpired:
169                 proc.kill()
170             log.close()
171

```

## 20. assistant (2026-06-22T01:26:57.676Z)

Now let me check the install/uninstall scripts to verify the pure functions tested actually exist and match, plus check for any concerns.

## 21. user (2026-06-22T01:26:58.377Z)

```

1     #!/usr/bin/env python3
2     """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
P2b).
3
4     Cross-platform (Windows / macOS / Linux), no native component (O1):
5
6     * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7     env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8     * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9     ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10    (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11    starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12    launcher (not by the engine) roots the ``.env`` in the app-home;
13    * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15    The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16    ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17    this script only wires the install + the launcher.
18
19    Usage::
20

```

```

21         python scripts/install.py                                # install '.' (this checkout), uv-
if-present else pip
22         python scripts/install.py --target badminton-highlight-indexer # from PyPI (once
published)
23         python scripts/install.py --method pip                    # force pip
24         python scripts/install.py --dry-run                        # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)
-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
exists)."""
51         system = system or platform.system()
52         e = os.environ if env is None else env
53         if system == "Windows":
54             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
55         elif system == "Darwin":
56             base = str(Path.home() / "Library" / "Application Support")
57         else:
58             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
59         return Path(base) / app_name
60
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
63         """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.
64
65         The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so
it's testable.
66         """
67         home = str(app_home)
68         if system == "Windows":
69             content = (
70                 "@echo off\r\n"
71                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
72                 f'set "RALLY_APP_HOME={home}"\r\n'
73                 f'{indexer_cmd} --app\r\n'
74             )
75             return "KhelSutra.bat", content, False
76         if system == "Darwin":
77             content = (
78                 "#!/bin/bash\n"

```

```

79         "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
80         f'export RALLY_APP_HOME="{home}"\n'
81         f"exec {indexer_cmd} --app\n"
82     )
83     return "KhelSutra.command", content, True
84 # Linux / other
85 content = (
86     "#!/bin/bash\n"
87     "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
88     f'export RALLY_APP_HOME="{home}"\n'
89     f"exec {indexer_cmd} --app\n"
90 )
91 return "khelsutra.sh", content, True
92
93
94 def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
95                   timestamp: str) -> dict:
96     """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
remove);
97     ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
98     return {
99         "schema": 1,
100         "product": "KhelSutra highlight indexer",
101         "package": PACKAGE,
102         "install_method": method,          # "uv" | "pip"
103         "indexer_command": "indexer",
104         "app_home": str(app_home),
105         "shortcut": str(shortcut),
106         "created": [str(p) for p in created],
107         "user_data": [
108             str(app_home / "config.json"),
109             str(app_home / ".env"),
110             str(app_home / "output"),
111         ],
112         "timestamp": timestamp,
113     }
114
115
116 def resolve_method(requested: str, uv_present: bool) -> str:
117     """Pick the installer: explicit beats auto; auto prefers uv when present, else
pip."""
118     if requested in ("uv", "pip"):
119         return requested
120     return "uv" if uv_present else "pip"
121
122
123 def install_command(method: str, target: str) -> list[str]:
124     """The argv for the chosen installer (PURE)."""
125     if method == "uv":
126         return ["uv", "tool", "install", target]
127     return [sys.executable, "-m", "pip", "install", target]
128
129
130 # --- IO / orchestration
-----
131
132 def _run(argv: list[str]) -> None:
133     print(f" $ {' '.join(argv)}")
134     subprocess.run(argv, check=True)
135
136
137 def main(argv: list[str] | None = None) -> int:
138     ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer

```

```

(LIGHT tier).")
139     ap.add_argument("--target", default=".",
140                     help="What to install: '.' (this checkout, default) or a PyPI
name/spec.")
141     ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
142                     help="Installer to use (default: uv if present, else pip).")
143     ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
144     args = ap.parse_args(argv)
145
146     system = platform.system()
147     app_home = default_os_app_home()
148     method = resolve_method(args.method, shutil.which("uv") is not None)
149     cmd = install_command(method, args.target)
150     fname, content, executable = shortcut_spec(system, app_home)
151     shortcut = app_home / fname
152
153     print(f"KheISutra installer ? {PACKAGE} (LIGHT, torch-free)")
154     print(f" OS           : {system}")
155     print(f" installer    : {method}")
156     print(f" target       : {args.target}")
157     print(f" app-home     : {app_home}")
158     print(f" shortcut     : {shortcut}")
159     if args.dry_run:
160         print("\n[dry-run] would run:")
161         print(f" $ {' '.join(cmd)}")
162         print(f" write shortcut -> {shortcut}")
163         print(f" write manifest -> {app_home / MANIFEST_NAME}")
164         print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
165         return 0
166
167     print("\n[1/3] installing the package?")
168     try:
169         _run(cmd)
170     except (subprocess.CalledProcessError, FileNotFoundError) as e:
171         print(f"[ERROR] install failed: {e}")
172         if method == "uv":
173             print(" (uv not usable? re-run with --method pip)")
174         return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KheISutra:")
200     if system == "Windows":

```

```

201         print(f"    double-click {shortcut}    (or run `indexer --app`)")
202     else:
203         print(f"    {shortcut}    (or run `indexer --app`)")
204     print("    The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f"    To uninstall: python \"{dst_uninstall}\"    (add --purge to also delete
your data)")
206     return 0
207
208
209     if __name__ == "__main__":
210         raise SystemExit(main())
211

```

## 22. user (2026-06-22T01:27:01.475Z)

```

1      #!/usr/bin/env python3
2      """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4      Reverses ``scripts/install.py`` from the manifest it wrote:
5
6      * uninstalls the package (`uv tool uninstall` / `pip uninstall -y` ? matches the
install method);
7      * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8      * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9      and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11      Usage::
12
13          python uninstall.py                # find the manifest in the app-home, keep user
data
14          python uninstall.py --purge        # also delete config.json / .env / output (your
data)
15          python uninstall.py --manifest /path/to/uninstall_manifest.json
16          python uninstall.py --dry-run      # print the plan, change nothing
17
18      stdlib-only; safe to run from the copy the installer dropped in the app-home.
19      """
20      from __future__ import annotations
21
22      import argparse
23      import json
24      import os
25      import platform
26      import shutil
27      import subprocess
28      import sys
29      from pathlib import Path
30
31      APP_DIR_NAME = "Khelsutra"
32      MANIFEST_NAME = "uninstall_manifest.json"
33
34
35      def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                             env: dict[str, str] | None = None) -> Path:
37          """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
given)."""
38          system = system or platform.system()
39          e = os.environ if env is None else env
40          if system == "Windows":
41              base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
42          elif system == "Darwin":
43              base = str(Path.home() / "Library" / "Application Support")

```



```

44         else:
45             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
46             return Path(base) / app_name
47
48
49     def find_manifest(explicit: str | None) -> Path | None:
50         """The manifest path: explicit arg, else next to THIS file, else the default app-
51         home."""
52         if explicit:
53             p = Path(explicit)
54             return p if p.exists() else None
55         for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
56                     default_os_app_home() / MANIFEST_NAME):
57             if cand.exists():
58                 return cand
59         return None
60
61     def uninstall_command(method: str, package: str) -> list[str]:
62         """The argv to remove the package (PURE)."""
63         if method == "uv":
64             return ["uv", "tool", "uninstall", package]
65         return [sys.executable, "-m", "pip", "uninstall", "-y", package]
66
67
68     def _rm(path: Path, dry: bool) -> None:
69         try:
70             if path.is_dir() and not path.is_symlink():
71                 print(f" - rmtree {path}")
72                 if not dry:
73                     shutil.rmtree(path, ignore_errors=True)
74             elif path.exists() or path.is_symlink():
75                 print(f" - remove {path}")
76                 if not dry:
77                     path.unlink(missing_ok=True)
78         except OSError as e:
79             print(f" ! could not remove {path}: {e}")
80
81
82     def main(argv: list[str] | None = None) -> int:
83         ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
84         indexer.")
85         ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
86         ap.add_argument("--purge", action="store_true",
87         help="Also delete YOUR data (config.json, .env, output/). Default
88         keeps it.")
89         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
90         nothing.")
91         args = ap.parse_args(argv)
92
93         manifest_path = find_manifest(args.manifest)
94         if not manifest_path:
95             print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
96             return 1
97         manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
98         method = manifest.get("install_method", "pip")
99         package = manifest.get("package", "badminton-highlight-indexer")
100         created = [Path(p) for p in manifest.get("created", [])]
101         user_data = [Path(p) for p in manifest.get("user_data", [])]
102         app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
103
104         print(f"KhelSutra uninstaller ? {package} (installed via {method})")
105         print(f" manifest : {manifest_path}")
106         print(f" app-home : {app_home}")
107         print(f" data      : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use"}

```

```

--purge to delete}')")
105     dry = args.dry_run
106
107     print("\n[1/3] removing the package?")
108     cmd = uninstall_command(method, package)
109     print(f" $ {' '.join(cmd)}")
110     if not dry:
111         try:
112             subprocess.run(cmd, check=False)
113         except FileNotFoundError as e:
114             print(f" ! {method} not found ({e}); remove the package manually.")
115
116     print("\n[2/3] removing installer-created files?")
117     # Remove the manifest LAST (we still reference it); skip the running uninstaller
copy itself.
118     self_path = Path(__file__).resolve()
119     for p in created:
120         if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
121             continue
122         _rm(p, dry)
123
124     if args.purge:
125         print("\n[3/3] purging user data (--purge)?")
126         for p in user_data:
127             _rm(p, dry)
128     else:
129         print("\n[3/3] keeping your data:")
130         for p in user_data:
131             print(f" · {p}")
132
133     # Manifest + (optionally) the empty app-home last.
134     _rm(manifest_path, dry)
135     if args.purge and app_home.exists():
136         try:
137             remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
138             if not remaining:
139                 _rm(app_home, dry)
140         except OSError:
141             pass
142
143     print("\n? Uninstalled." + (" " if args.purge else " Your data was kept. "))
144     if self_path.parent == app_home:
145         print(f" (you can delete this script + the app-home folder manually:
{app_home})")
146     return 0
147
148
149     if __name__ == "__main__":
150         raise SystemExit(main())
151

```

### 23. assistant (2026-06-22T01:27:05.166Z)

Now let me look at the docs changes (\$7/\$8/\$9 + changelog) to verify 08 resolution is honestly recorded.

### 24. user (2026-06-22T01:27:06.441Z)

```

diff --git a/docs/PACKAGING_PLAN.md b/docs/PACKAGING_PLAN.md
index 94788dd..c40d04b 100644
--- a/docs/PACKAGING_PLAN.md
+++ b/docs/PACKAGING_PLAN.md
@@ -12,9 +12,9 @@
**You are here (2026-06-22):** `origin/master` has the **LIGHT-tier engine fully bundle-ready**
? `pip install` ? `indexer --app` boots and serves the real KhelSutra SPA single-origin,

```

**\*\*torch-free\*\***, with resolved app-home state, an env-overridable ffmpeg, DNS-rebinding-guarded localhost binding, single-instance locking, and safe DB upgrades. 11 PRs merged; 0 open PRs; 0 GCP VMs (\$0). Ramp up via this doc's \$7 tracker + memory `[[packaging-app-plan]]`.

**\*\*Remaining work, in priority order (each its own small PR; branch off `origin/master`, explicit paths ? shared checkout):\*\***

-1. **\*\*P3 GPU validation + screencast (owner wants first; ~\$0.50, well under the \$100 cap).\*\***  
Provision an L4 (`deploy/gcp/create\_gpu\_vm.sh`) ? install the **\*\*wheel\*\*** on the fresh GPU box + the WASB env ? validate the packaged stack end-to-end (`indexer --app` ? bundled SPA ? native-WASB ? reel) ? **\*\*screencast\*\*** ? `teardown.sh` (deletes) + confirm \$0. ? Run as a FOCUSED op (billable VM ? teardown is priority #1). ? Known blocker: native-WASB returned **\*\*0 rallies\*\*** on a fresh build last session ? reproducing the 22-rally result needs the engine session's exact tuned `worker infer detector\_impl=native` config. Confirm cmd + cost before spend; `[[gcp-vm-always-delete]]`.

-2. **\*\*P2b-install\*\*** (\$7) ? `uv tool install` / a generated cross-platform install script (now a clean **\*\*torch-free\*\*** LIGHT install thanks to P2e) + ship a truly-local default `config.json` + an uninstall manifest.

-3. **\*\*P2c first-run wizard\*\*** (\$7) ? the 3-step wizard UI (Gemini key / compute / videos) in the **\*\*khelsutra `web/` SPA\*\***, rendered from `GET /api/capabilities`. **\*\*Cross-repo\*\*** (`avidullu/khelsutra`) ? coordinate like P1b (the SPA is gitignored-dist; refresh `backend/ui/` via `scripts/bundle\_ui.sh` after).

+1. ? **\*\*P3 GPU validation ? DONE (2026-06-22, \$0).\*\*** Validated the packaged **\*\*wheel\*\*** end-to-end on a real L4: torch-free wheel install ? `indexer --app` ? bundled SPA ? native-WASB (separate `\$WASB\_PYTHON` conda env) ? **\*\*22 rallies ? 73.7 MB watermarked reel\*\***, all via the REAL API. Proof in `gs://khelsutra-rally-corpus/highlights/p3\_{packaging\_highlights.mp4,run.log,app.log}`. VM deleted, \$0 (~\$0.17, ~14 min). The old "0 rallies" blocker was already solved by `skip\_ai\_handoff=true` (`docs/CLOUD\_GPU\_DRYRUN\_GUIDE.md`). Screencast = owner records from the preserved reel/logs.

+2. ? **\*\*P2b-install ? DONE (2026-06-22).\*\*** Engine seeds a truly-local + `motion` (gate **\*\*08\*\***) torch-free default config on the first `indexer --app` (never overwrites); cross-platform `scripts/install.py` (`uv tool install` else pip) + per-OS launcher shortcut that pins `RALLY\_APP\_HOME` + `scripts/uninstall.py` driven by an install manifest (keeps user data unless `--purge`). See \$7.

+3. **\*\*? NEXT ? P2c first-run wizard\*\*** (\$7) ? the 3-step wizard UI (Gemini key / compute / videos) in the **\*\*khelsutra `web/` SPA\*\***, rendered from `GET /api/capabilities`. **\*\*Cross-repo\*\*** (`avidullu/khelsutra`) ? coordinate like P1b (the SPA is gitignored-dist; refresh `backend/ui/` via `scripts/bundle\_ui.sh` after).

4. Then **\*\*P3-cloud\*\*** (Cloud Run dispatch, micromamba Dockerfile ? P3a) and **\*\*P4\*\*** (ONNX export ? dissolve the dual-env; training + in-app annotation = the **\*\*north-star\*\*** completion).

---

@@ -110,7 +110,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **\*\*One small PR per \*\*?** P1 COMPLETE (P1a #300 + P1b #302) ? the engine now serves the real KhelSutra SPA single-origin from the wheel, with the console/wizard endpoints in place. ? NEXT = P2\*\* (cross-platform launcher `indexer --app` + first-run wizard rendered from `/api/capabilities`). P2's wizard UI is SPA work (khelsutra repo) ? coordinate like P1b.

| **\*\*P2a\*\*** | **\*\*ffmpeg/ffprobe resolver\*\*** (`backend/utils/ffmpeg.py`: `ffmpeg\_bin`/`ffprobe\_bin` env-overridable via `RALLY\_FFMPEG`/`RALLY\_FFPROBE` + `require\_ffmpeg` fail-loud w/ per-OS hint) wrapping all 13 argv sites; byte-identical when unset. \*(first-run cpu-mock self-test deferred to the wizard slice.)\* | ? | No | ? | [305](https://github.com/Khelsutra/badminton-highlight-indexer/pull/305) |

| P2b-launcher | **\*\*Pure-Python `indexer --app` launcher\*\*** ? (Win/Mac/Linux, no native component): localhost-only bind + free-port fallback + health-poll ? `webbrowser.open` (`RALLY\_NO\_BROWSER` escape); never CLI single-shot; rides the P0e lock + P0a app-home. | P1a | No | ? | [303](https://github.com/Khelsutra/badminton-highlight-indexer/pull/303) |

-| P2b-install | **\*\*Install path\*\*** (`uv tool install` / a generated cross-platform install script; ship `config.json` defaulting truly-local; optional thin per-OS shortcut to `indexer --app`) | P2b-launcher | No (signing only if clickable binaries ? 09) | ? | ? |

+| **\*\*P2b-install\*\*** | **\*\*Install path + first-run default + uninstall.\*\*** Engine seeds a truly-local + `motion` (gate **\*\*08\*\***) torch-free default `config.json` on the first `indexer --app` (`write\_light\_default\_config`; minimal, profile-driven, NEVER overwrites). `scripts/install.py` (cross-platform, stdlib: `uv tool install` else pip + a per-OS launcher shortcut that pins `RALLY\_APP\_HOME` ? load-bearing since `load\_dotenv` runs at import + an uninstall manifest). `scripts/uninstall.py` (manifest-driven; keeps user data unless `--purge`). README "batteries-included install". | P2b-launcher | No (signing only if clickable binaries ? 09) | ? |

```
[#310](https://github.com/Khelsutra/badminton-highlight-indexer/pull/310) |
| P2c | **In-UI 3-step first-run wizard** (Gemini key / Compute / Videos), rendered from
`/api/capabilities`, en/kn/hi | P1b | No | ? | ? |
| P2d | **DB upgrade contract** ? `SCHEMA_VERSION` + downgrade guard (`SchemaTooNewError`) +
backup-before-migrate (`<db>.bak-v<old>`, WAL-complete) + `backup_database()` export. Byte-
identical for fresh/current DBs. *(uninstall manifest deferred to P2b-install.)* | ? | No | ? |
[#307](https://github.com/Khelsutra/badminton-highlight-indexer/pull/307) |
| P2e | **Lazy-torch for the LIGHT tier** ? deferred the one eager `import torch`
(`person_detector.py`) so `import backend.main` works WITHOUT torch (the LIGHT-tier promise,
enforced by a blocked-torch subprocess test). | ? | No | ? |
[#308](https://github.com/Khelsutra/badminton-highlight-indexer/pull/308) |
@@ -130,11 +130,11 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
5. **05 ? Training in v1 UI?** Recommend defer (synth=mock, real=GPU+dual-env, `ship=False`).
6. **06 ? Annotation in v1?** in-browser HTML5 annotator (recommended) vs bundle VLC vs defer.
7. **07 ? Ship AI-drafted kn/hi to real users before native-speaker review, or gate the
trilingual UI on review?**
-8. **08 ? Shipped DEFAULT `config.json` / profile** = truly-local (so the box never surprise-
bills on Gemini nor returns 0 rallies on WASB-without-key)?
+8. **08 ? RESOLVED 2026-06-22:** the shipped DEFAULT (seeded on the first `indexer --app`) =
**profile `truly-local` + detector `motion` + `use_gpu` false** ? a torch-free, offline, zero-
setup experience that never surprise-bills on Gemini nor returns 0 rallies on WASB-without-a-
key. The P2c wizard upgrades to Gemini once a key is supplied. (Implemented in P2b-install via
`write_light_default_config`.)
9. **09 ? Code-signing budget** (Windows Authenticode now, macOS Developer-ID later) so non-
tech users don't hit SmartScreen/AV prompts.
```

```
## 9. Definition of done
-- [ ] **01/02/08 decided in writing** (scope + delivery shape + default profile).
+- [x] **01/02/08 decided in writing** (scope + delivery shape + default profile) ? 01/02
(2026-06-21), **08 (2026-06-22): truly-local + `motion`**.
- [ ] P0a?P0e merged: launching `indexer --server` from **any** CWD finds the same config/DB;
`/api/config` leaks nothing; all CI lanes (incl. the new license-scan + built-artifact lanes)
green at the coverage floor.
- [ ] P1a/P1b merged: FastAPI serves the **real SPA** single-origin with refresh-survival;
capabilities/videos/reels return real data; contract test guards SPA?engine skew.
- [ ] P2a?P2d merged: a **clean Windows VM ? installer ? reel downloaded (en+hi)** with **no
Python/git/ffmpeg pre-installed**; uninstall is clean; upgrade preserves data.
@@ -167,3 +167,4 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
### Changelog
- 2026-06-21 ? Created from an exhaustive research workflow (29 agents) + per-claim re-
verification at file:line.
- 2026-06-21 ? Owner resolved 01 + 02: **cross-platform (Win/Mac/Linux) webserver, no native
component** (L9); **LIGHT-first with a WASB-complete north star as the sole completion goal**
(L10). Updated $0/$2/$4.5/$6/$7-P2b/$8/$9 accordingly. Remaining gates: 03 (WASB-C3 counsel), 04
(hosted vs BYO-cloud), 05?09.
+- 2026-06-22 ? **P3 GPU validation DONE** ($0, L4): the packaged wheel ran end-to-end (torch-
free wheel ? `indexer --app` ? bundled SPA ? native-WASB ? 22 rallies ? 73.7 MB reel).
**P2b-install DONE**: owner resolved **08 = truly-local + `motion`**; engine seeds it on first
`--app` run + `scripts/install.py`/`uninstall.py` + README. $7 P2b ?, $8 08 + $9 08 box
resolved. Remaining gates: 03, 04, 05?07, 09.
```

## 25. assistant (2026-06-22T01:27:10.587Z)

The 08 resolution is recorded consistently in \$7, \$8, \$9, and the changelog, and it matches the code (`truly-local` + `motion` + `use\_gpu` false). Now let me verify the byte-identity claim for `--server`/CLI. The key concern: is `args.app` the only gate, and is there any unconditional side effect? Let me look at the full main() flow around the seeding and the server/CLI branches.

## 26. user (2026-06-22T01:27:11.663Z)

```
900         if not args.video_path:
901             print("[ERROR] Please provide --video-path to extract a segment.")
902         return
903
904         # Determine output path
```

```

905         out_filename = args.trim_output or "trimmed_segment.mp4"
906         if os.path.isabs(out_filename):
907             out_path = out_filename
908             out_dir = os.path.dirname(out_path)
909             out_name = os.path.basename(out_path)
910         else:
911             out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-
identical when unset)
912             out_path = os.path.join(out_dir, out_filename)
913             out_name = out_filename
914
915         os.makedirs(out_dir, exist_ok=True)
916         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
917
918         # global_config isn't initialized this early; load the typed config so the trim
919         # clip carries the same configured stitching paddings + watermark as a real
920         # highlight (watermark default-on).
921         stitching = load_app_config().stitching
922         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
923                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
924         try:
925             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
926             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
927         except Exception as e:
928             print(f"[ERROR] Failed to compile trimmed segment: {e}")
929         return
930
931     if args.debug_clip is not None:
932         if not args.video_path:
933             print("[ERROR] Please provide --video-path to debug a specific clip.")
934             return
935         run_cli_diagnose_clip(args.video_path, args.debug_clip)
936         return
937
938     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
`--app`
939     # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
940     # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
941     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
942     # it ? so a fresh `pip`/`uv` tool install + `indexer --app` just works. NEVER
overwrites an
943     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
944     # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
945     if args.app:
946         seeded_path, seeded = write_light_default_config()
947         if seeded:
948             print(f"[APP] First run: seeded a truly-local default config at
{seeded_path} "
949                 f"(detector 'motion' ? offline, no API key or GPU needed). "
950                 f"Use the in-app setup to switch to Gemini.")
951
952     # Initialize Global Config and DB
953     global global_config, db_instance
954     global_config = load_app_config() # returns typed AppConfig
955     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
956

```

```

957     # The CLI --output-dir overrides the configured output directory. It now lives
958     # on the typed model (OutputConfig.output_dir), so every consumer ? including
959     # /api/compile ? reads the same value instead of a write-only runtime hack.
960     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
961     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
962     global_config.output.output_dir = resolve_output_dir(args.output_dir)
963     os.makedirs(global_config.output.output_dir, exist_ok=True)
964
965     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
966
967     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
968     # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
969     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
970     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
971     global _instance_lock
972     try:
973         _instance_lock = acquire_lock(db_path + ".lock")
974     except OSError as exc:
975         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
976         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
977         # legitimate single instance).
978         logger.warning("[JOBS] Could not create the single-instance lock at %s.lock
(%s); proceeding "
979                        "WITHOUT the second-instance guard.", db_path, exc)
980         _instance_lock = None
981     else:
982         if _instance_lock is None:
983             print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
984                  f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
985                  f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
986             sys.exit(1)
987
988     db_instance = Database(db_path)
989
990     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
991     # the executor is in-process. Mark them failed so pollers see a terminal state.
992     swept = db_instance.fail_stale_jobs()
993     if swept:
994         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
995
996     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
997     if args.video_path and not args.server and not args.app:
998         print(f"[CLI] Processing video file: {args.video_path}")
999     # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.

```

## 27. assistant (2026-06-22T01:27:16.310Z)

The seeding is correctly gated on `if args.app:` and placed before `load\_app\_config()`. The `--server` and CLI single-shot paths never reach the seeding. Byte-identity is preserved for those paths.

Now let me verify two things: (1) that the seeding fires before any return path that the CLI debug/trim modes use (they `return` earlier, so they're untouched ? good), and (2) whether there's a concern about the `--app` test isolation. Let me check `default\_config\_path` and `RALLY\_APP\_HOME` resolution.

## 28. user (2026-06-22T01:27:17.873Z)

```
10: * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher sets
it).
11: * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a behaviour.
15: calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means
importing
30: APP_HOME_ENV = "RALLY_APP_HOME"
33: APP_DIR_NAME = "KheIsutra"
36: def app_home() -> Path:
39:   ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD, byte-
identical
53: def default_config_path() -> Path:
56:   With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative), unchanged.
70:   Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file exists.
With the
84:   With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default ``"output"``
when
101: def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
103:   ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
```

## 29. assistant (2026-06-22T01:27:21.976Z)

Good. The test `test\_app\_mode\_seeds\_truly\_local\_config` sets `RALLY\_APP\_HOME` to an isolated tmp dir, so `default\_config\_path()` roots there. That's correctly isolated. And `test\_app\_mode\_boots\_and\_serves\_bundled\_ui` does NOT set `RALLY\_APP\_HOME` and runs with `cwd=REPO\_ROOT` ? meaning the `--app` seeding would write a `config.json` into the repo root if one doesn't exist. Let me check whether that's a concern.

## 30. user (2026-06-22T01:27:23.435Z)

```
-rw-r--r-- 1 avidu 197609 2735 Jun 22 04:00 config.json
---gitignore check---
exit=1
---is config.json tracked?---
config.json
```

## 31. assistant (2026-06-22T01:27:29.656Z)

`config.json` is tracked in the repo. So in `test\_app\_mode\_boots\_and\_serves\_bundled\_ui` (no `RALLY\_APP\_HOME`, `cwd=REPO\_ROOT`), the `--app` seeding will find an existing `config.json` and NOT overwrite it (correct no-overwrite behavior). But this raises a concern: if a developer's checkout did NOT have `config.json` (it's a tracked file, so it normally exists), the test would write one. Since it's tracked, it always exists ? but if a contributor's working tree had it deleted/modified, the boot-and-serve test could leave a seeded file. This is minor. Let me check the existing tracked config.json content to understand whether the boot-test relies on it.

## 32. user (2026-06-22T01:27:30.105Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
```

```

11         "relevance": {
12             "court_green_lower": [
13                 35,
14                 40,
15                 40
16             ],
17             "court_green_upper": [
18                 85,
19                 255,
20                 255
21             ],
22             "court_pixels_min_ratio": 0.05
23         }
24     },
25     "indexing": {
26         "default_segmenter": "gemini",
27         "use_gpu": true,
28         "motion_threshold": 0.08,
29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
62             SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
63             "sport": "badminton",
64             "wsl_stage_dir": "~/clips",
65             "timeout_sec": 1800,
66             "velocity_threshold": 0.02
67         }
68     },
69     "output": {
70         "default_highlights_name": "highlights.mp4",
71         "db_name": "sports_indexer.db"
72     },
73     "stitching": {
74         "begin_padding": 0.5,
75         "end_padding": 0.5,

```



```

75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

### 33. assistant (2026-06-22T01:27:33.297Z)

Now let me run the new tests to verify they actually pass and aren't flaky. Let me run the fast unit tests first, then the subprocess tests.

### 34. user (2026-06-22T01:27:36.692Z)

```

..... [100%]
14 passed in 0.33s

```

### 35. user (2026-06-22T01:27:51.107Z)

```

..... [100%]
8 passed in 9.45s

```

### 36. assistant (2026-06-22T01:28:03.770Z)

Both pass. Now let me scrutinize the potential flakiness concern in `test\_app\_mode\_seeds\_truly\_local\_config`. The test polls `cfg.exists()` first, then checks `proc.poll()`. There's a subtle race: if the process seeds the file AND exits before the test loop runs, the `cfg.exists()` check would still pass (file persists), so that's fine. But what if the process crashes BEFORE seeding? Then `proc.poll()` catches it. The ordering (check file first, then poll) is correct? it avoids a false-fail where the process is healthy but the file check hasn't been re-pollled.

Let me verify one more potential issue: does the `--app` server in `test\_app\_mode\_seeds\_truly\_local\_config` actually start cleanly with the seeded `truly-local` config, given `enforce\_startup\_checks`? The `truly-local` profile with `storage.backend=local` should pass. Let me check the startup enforcement doesn't reject motion+truly-local, and confirm the test's `cwd=str(tmp\_path)` plus `RALLY\_APP\_HOME` interplay for the output dir.

### 37. user (2026-06-22T01:28:09.636Z)

```

180         return checks # no profile ? only the exposure posture (itself off unless edge
auth is on).
181
182         storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
183         indexing = getattr(config, "indexing", None)
184         detector_impl = getattr(indexing, "detector_impl", "auto")
185         device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
186
187         if profile == "truly-local":
188             # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
189             # an inconsistent (but not fatal) choice worth surfacing.
190             if storage_backend not in ("local", "gdrive"):
191                 checks.append(StartupCheck(
192                     LEVEL_WARN, "STORAGE_UNUSUAL",
193                     f"profile 'truly-local' usually uses local/gdrive storage, but
storage_backend="
194                     f'"{storage_backend}"'. Reads will go to a remote backend.",

```

```

195         ))
196         # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
197         # best-effort and may be False even when a subprocess detector env has CUDA).
198         if cuda_available is False:
199             if device == "cuda":
200                 checks.append(StartupCheck(
201                     LEVEL_WARN, "CUDA_NOT_VISIBLE",
202                     "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
203                     "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
204                     "'auto' for the CPU fallback (M4).",
205                 ))
206             elif device == "auto":
207                 checks.append(StartupCheck(
208                     LEVEL_WARN, "CPU_FALLBACK",
209                     "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
210                     "to CPU (much slower). If torch lives only in the detector
subprocess env this "
211                     "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
212                 ))
213
214         elif profile == "cloud-serving":
215             # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
216             if storage_backend not in _CLOUD_STORAGE:
217                 checks.append(StartupCheck(
218                     LEVEL_ERROR, "STORAGE_INCOHERENT",
219                     f"profile 'cloud-serving' requires a cloud storage backend "

```

### 38. assistant (2026-06-22T01:28:14.755Z)

Good ? for `truly-local` with `storage.backend=local`, these are all WARN-level at most (no ERROR), so `\_enforce\_startup\_or\_exit` won't abort. The seeded config boots cleanly. The tests passing confirms this.

Let me double-check the `\_enforce\_startup\_or\_exit` only exits on ERROR-level checks (not WARN), to be sure the seeded config never blocks startup.

### 39. user (2026-06-22T01:28:16.663Z)

```

846: def _enforce_startup_or_exit(config) -> None:
847:     """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
BEFORE
858:         sys.exit(1)
955:     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile
986:         sys.exit(1)
---
846: def _enforce_startup_or_exit(config) -> None:

```

### 40. user (2026-06-22T01:28:20.520Z)

```

846     def _enforce_startup_or_exit(config) -> None:
847         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
848             any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
849             the exposed server (`include_exposure=True`, M9 hardening), a half-configured
exposure (edge
850             auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no

```

```

851         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
852         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
853         fail-fast wiring + the default-OFF no-op are unit-testable.""
854         try:
855             enforce_startup_checks(config, logger=logger, include_exposure=True)
856         except StartupCheckError as e:
857             logger.error("[STARTUP] refusing to start: %s", e)
858             sys.exit(1)
859
860
861     def main():
862         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
863         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
864         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
865         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")

```

#### 41. assistant (2026-06-22T01:28:24.868Z)

Confirmed ? only ERROR/FATAL findings exit; the `truly-local` seeded config produces at most WARNs.

Now let me verify the broader regression concern: run the full test suite focused on config/loader/startup and the existing app/launcher tests, to make sure nothing in this PR broke existing behavior.

#### 42. user (2026-06-22T01:28:53.495Z)

```

..... [ 34%]
..... [ 69%]
..... [100%]
206 passed, 1148 deselected in 25.26s

```

#### 43. assistant (2026-06-22T01:28:57.119Z)

All passing. Now let me verify lint/type cleanliness for the changed files, as the pre-LGTM gate requires.

#### 44. user (2026-06-22T01:28:58.403Z)

All checks passed!

#### 45. user (2026-06-22T01:29:04.020Z)

Success: no issues found in 3 source files

#### 46. assistant (2026-06-22T01:29:07.955Z)

Now let me scrutinize a subtle inconsistency I noticed. The README claims and the install script behavior around the `motion` segmenter ? let me verify `motion` is a real registered segmenter (not a typo), since if `motion` isn't a valid segmenter, the seeded config would break at runtime.

#### 47. user (2026-06-22T01:29:09.511Z)

```

backend/pipeline/segmenters/motion.py:14:         return "motion"
backend/config/models.py:128:     default_segmenter: str = "yolo_hybrid"
backend/config/models.py:210:     @field_validator("default_segmenter")

```

#### 48. user (2026-06-22T01:29:12.260Z)

```
205
206     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208     fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210     @field_validator("default_segmenter")
211     @classmethod
212     def segmenter_must_be_registered(cls, v: str) -> str:
213         # Registry check happens at runtime in main/segmenter selection.
214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
```

#### 49. assistant (2026-06-22T01:29:15.490Z)

`motion` is a real segmenter (`backend/pipeline/segmenters/motion.py` returns `"motion"`). And it's torch-free (pure-OpenCV), which validates the LIGHT-tier / torch-free claim. Let me confirm motion has no torch import.

#### 50. user (2026-06-22T01:29:16.787Z)

```
1:import cv2
exit=0
```

#### 51. assistant (2026-06-22T01:29:19.887Z)

Motion is pure-OpenCV, torch-free ? confirms the LIGHT/torch-free claim and the L-decision (LIGHT torch-free).

Now let me check the README diff to make sure its claims are honest and consistent.

#### 52. user (2026-06-22T01:29:20.723Z)

```
diff --git a/README.md b/README.md
index 674e304..0bd46dd 100644
--- a/README.md
+++ b/README.md
@@ -32,6 +32,37 @@ The pipeline is built around three pluggable registries ? **validators**,
**se

## ?? Setup & Diagnostics
```

```

+### 0. Batteries-included install (LIGHT tier ? recommended for end users)
+
+For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform
+installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock), wires a
+desktop launcher, and seeds a sensible default config.
+
+```bash
+# Preferred ? uv puts `indexer` on your PATH in an isolated tool env:
+uv tool install badminton-highlight-indexer          # (once published) or:  uv tool install .
+
+# Or the cross-platform installer script (uv-if-present, else pip) ? also creates a launcher
+# shortcut + an uninstall manifest:
+python scripts/install.py                          # add --method pip to force pip, --dry-run
to preview
+```
+
+Then launch the app (the installer's shortcut, or directly):
+
+```bash
+indexer --app          # boots a local server (127.0.0.1) and opens the bundled Khelsutra UI
+```
+
+* **First run seeds a truly-local default** ? detector `motion` (offline, **no API key, no GPU,
+ no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels)
lives
+ in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` /
+ `~/local/share/Khelsutra`) when launched via the installer's shortcut.
+* **FFmpeg is still required** (see step 1) ? it is intentionally not redistributed (GPL
`libx264`).
+* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your
config/reels
+ by default; add `--purge` to delete them too.
+
+The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see
`docs/PACKAGING_PLAN.md`.
+
+### 1. External System Requirements
+The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes,
and stitch clips.

```

### 53. assistant (2026-06-22T01:29:30.451Z)

README is honest and consistent. Now let me think harder about the test-quality and flakiness questions, plus one real edge case in the install/uninstall scripts.

I noticed in `uninstall.py` `main()`, the `app\_home` empty-dir cleanup at the end: it filters out `self\_path` but the uninstaller copy lives at `app\_home / "uninstall.py"`, and the manifest was already removed via `\_rm(manifest\_path)`. The `created` list includes `uninstall.py` (the copied uninstaller), which is skipped in the removal loop (`p.resolve() == self\_path`). So after uninstall, `uninstall.py` itself remains in app\_home, meaning `remaining` will contain it and the rmtree won't fire on `--purge` ? but the code prints a manual-delete hint. This is intended behavior, not a bug.

Let me verify one concrete concern about test depth: `test\_install\_uninstall\_roundtrip\_manifest` asserts `uninstall\_command()[0:3] in (["uv",...], [sys.executable, "-m", "pip"])`. This is checking against `install\_mod.sys.executable`. Let me confirm the pip branch is actually exercised meaningfully ? the test uses `method="pip"`, so the roundtrip resolves to the pip argv. That's a real assertion.

Now let me check whether the `test\_app\_mode\_boots\_and\_serves\_bundled\_ui` test (which runs with `cwd=REPO\_ROOT`, no `RALLY\_APP\_HOME`) could leave a stray `config.json` if the tracked one were absent, and whether the test relies on the seeding at all (it does NOT assert seeding). Let me confirm this test predates the PR (was it modified?).

### 54. user (2026-06-22T01:29:34.172Z)

```

diff --git a/tests/test_app_launcher.py b/tests/test_app_launcher.py
index 17640ba..78434c0 100644
--- a/tests/test_app_launcher.py
+++ b/tests/test_app_launcher.py
@@ -126,3 +126,45 @@ def test_app_mode_boots_and_serves_bundled_ui(tmp_path):
     except subprocess.TimeoutExpired:
         proc.kill()
         log.close()
+
+
+def test_app_mode_seeds_truly_local_config(tmp_path):
+    """First `indexer --app` run with an EMPTY app-home seeds the truly-local 'motion' default
+    config (PACKAGING_PLAN.md P2b / gate 08) BEFORE serving ? proving the main() wiring
+    fires."""
+    import json
+    port = _free_port()
+    app_home = tmp_path / "apphome"
+    app_home.mkdir()
+    env = {
+        **os.environ,
+        "RALLY_NO_BROWSER": "1",
+        "RALLY_APP_HOME": str(app_home), # isolate: never touch the real OS app-data dir
+        "PYTHONPATH": REPO_ROOT + os.pathsep + os.environ.get("PYTHONPATH", ""),
+    }
+    log = open(tmp_path / "app.log", "wb")
+    proc = subprocess.Popen(
+        [sys.executable, "-m", "backend.main", "--app", "--port", str(port)],
+        cwd=str(tmp_path), env=env, stdout=log, stderr=subprocess.STDOUT,
+    )
+    cfg = app_home / "config.json"
+    try:
+        deadline = time.time() + 90
+        seeded = False
+        while time.time() < deadline:
+            if cfg.exists(): # check the file FIRST: seeding precedes (and is independent of)
serving
+                seeded = True
+                break
+            if proc.poll() is not None:
+                pytest.fail(f"--app exited before seeding (code {proc.returncode}); see
{tmp_path/'app.log'}")
+                time.sleep(0.4)
+            assert seeded, "config.json was not seeded under RALLY_APP_HOME"
+            data = json.loads(cfg.read_text(encoding="utf-8"))
+            assert data["indexing"]["default_segmenter"] == "motion"
+            assert data["deployment"]["profile"] == "truly-local"
+        finally:
+            proc.terminate()
+            try:
+                proc.wait(timeout=10)
+            except subprocess.TimeoutExpired:
+                proc.kill()
+            log.close()

```

## 55. assistant (2026-06-22T01:29:41.033Z)

The new seeding test is properly isolated via `RALLY\_APP\_HOME` to a tmp dir and uses `cwd=str(tmp\_path)` ? so it never touches the repo or the real OS app-data dir. Good isolation.

There's one subtle race in `test\_app\_mode\_seeds\_truly\_local\_config`: it reads `cfg.read\_text()` and `json.loads()` the moment `cfg.exists()` returns True. The engine writes the file atomically? Let me check: `write\_light\_default\_config` uses `cfg\_path.write\_text(...)` which is a single open-write-close. On most platforms a partial read is theoretically possible but the file is tiny (~100 bytes) and written in one `write\_text` call. In practice this won't be flaky for a file this small written in a single syscall-ish operation, and the test passed. It's a

theoretical nit, not a real concern.

Let me run the broader test suite once more to be thorough about regression ? specifically any test that might exercise CLI/server byte-identity or main().

#### 56. user (2026-06-22T01:29:52.378Z)

```
..... [100%]  
22 passed in 8.80s
```

#### 57. assistant (2026-06-22T01:29:54.933Z)

Let me run the install/uninstall scripts in `--dry-run` to confirm they actually execute end-to-end without error (the orchestration paths aren't unit-tested, only the pure helpers).

#### 58. user (2026-06-22T01:29:56.402Z)

```
KhelSutra installer ? badminton-highlight-indexer (LIGHT, torch-free)  
OS           : Windows  
installer    : pip  
target       : .  
app-home     : C:\Users\avidu\AppData\Roaming\Khelsutra  
shortcut     : C:\Users\avidu\AppData\Roaming\Khelsutra\KhelSutra.bat
```

[dry-run] would run:

```
$ C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pip install .  
write shortcut -> C:\Users\avidu\AppData\Roaming\Khelsutra\KhelSutra.bat  
write manifest -> C:\Users\avidu\AppData\Roaming\Khelsutra\uninstall_manifest.json  
copy uninstaller -> C:\Users\avidu\AppData\Roaming\Khelsutra\uninstall.py
```

#### 59. assistant (2026-06-22T01:30:02.601Z)

The installer dry-run works and resolves the Windows app-home correctly. Note `uv` is not present here (resolves to pip), consistent with `resolve\_method`.

One thing to verify regarding consistency with `backend.utils.app\_home.default\_os\_app\_home` ? the script claims to mirror it. Let me confirm the two implementations agree (a drift here would be a real bug: the launcher would pin a different RALLY\_APP\_HOME than the engine's own resolver expects).

#### 60. user (2026-06-22T01:30:02.677Z)

```
101 def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:  
102     """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as  
103     ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).  
104  
105     * Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)  
106     * macOS   ? ``~/Library/Application Support/<app_name>``  
107     * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/local/share/<app_name>`` if  
unset)  
108     """  
109     system = platform.system()  
110     if system == "Windows":  
111         base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")  
112     elif system == "Darwin":  
113         base = str(Path.home() / "Library" / "Application Support")  
114     else:  
115         base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")  
116     return Path(base) / app_name  
117
```

#### 61. assistant (2026-06-22T01:30:07.474Z)

The script's `default\_os\_app\_home` is a faithful mirror of the engine's

`backend.utils.app\_home.default\_os\_app\_home` ? same `APP\_DIR\_NAME = "Khelsutra"`, same per-OS bases, same precedence. They agree. The standalone duplication is justified in the docstring (the installer runs before the package exists). One minor maintenance risk: the duplication can drift, but there is no test asserting parity between the two implementations. That's a nit worth flagging.

Let me confirm `APP\_DIR\_NAME` matches exactly in both the engine and both scripts.

## 62. user (2026-06-22T01:30:08.606Z)

```
backend/utils/app_home.py:33:APP_DIR_NAME = "Khelsutra"
backend/utils/app_home.py:101:def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
scripts/install.py:41:APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
scripts/install.py:47:def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None =
None,
scripts/uninstall.py:31:APP_DIR_NAME = "Khelsutra"
scripts/uninstall.py:35:def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None =
= None,
```

## 63. assistant (2026-06-22T01:30:24.420Z)

All three agree on `"Khelsutra"`.

Now let me consolidate. I've verified the five scrutiny points. Let me summarize my assessment against each:

1. **\*\*L-decisions honored\*\***: LIGHT torch-free (motion = pure-OpenCV, no torch); no native component (stdlib-only cross-platform scripts + `webbrowser`); default-OFF for `--server`/CLI (seeding gated strictly on `if args.app:` , placed before `load\_app\_config()` , after all CLI-return paths). ?
2. **\*\*08 recorded honestly\*\***: \$7 row ? with PR #310, \$8 08 "RESOLVED 2026-06-22" with the exact triple, \$9 DoD checkbox flipped, changelog entry ? all matching the code (`LIGHT\_DEFAULT\_CONFIG` = `truly-local` + `motion` + `use\_gpu` false). ?
3. **\*\*Tests assert important behavior\*\***: no-overwrite idempotency (`test\_seed\_does\_not\_overwrite\_existing` preserves the user's `gemini` choice), effective motion config via the loader+preset expansion (`test\_seeded\_config\_loads\_to\_truly\_local\_motion` asserts `skip\_ai\_handoff=True` + `storage.backend=local` come from the preset), `--app` wiring (subprocess seeds under isolated `RALLY\_APP\_HOME`). These are substantive, not tautological. ?
4. **\*\*Flakiness\*\***: subprocess test is well-isolated (`RALLY\_APP\_HOME` to tmp, free port, `cwd=tmp\_path`, 90s deadline, checks file before poll). Theoretical partial-read of the tiny JSON is negligible. No real flakiness. ?
5. **\*\*Byte-identity for `--server`/CLI\*\***: seeding is strictly under `if args.app:` ; 206 config/startup/server tests pass; no unconditional new side effects. ?

Everything verified locally: 22 new tests pass, 206 related tests pass, ruff clean, mypy clean, dry-run works, motion is real and torch-free. No blocker/major/minor findings. Two nits worth recording (low value): the duplicated `default\_os\_app\_home` has no parity test, and the orchestration paths of install/uninstall (the IO sections) are not directly tested (only pure helpers + dry-run). These are minor/nit-level and consistent with the "pure-logic locally, machine-side to owner" convention.

I'll report these honestly as nits rather than inventing problems.

## 64. user (2026-06-22T01:30:35.838Z)

Structured output provided successfully

## 65. assistant (2026-06-22T01:30:51.510Z)

I've completed the consistency + regression review of PR #310. Summary of what I verified



locally (all green):

**\*\*Verdict: clean. No blocker/major/minor findings ? only two low-value nits.\*\***

The five scrutiny points all pass:

1. **\*\*L-decisions honored.\*\*** ``LIGHT_DEFAULT_CONFIG` = `truly-local` + `motion` + `use_gpu:false`.` ``motion`` is a real registered segmenter (``backend/pipeline/segmenters/motion.py``) and is pure-OpenCV (only ``import cv2``, no torch) ? the torch-free LIGHT promise holds. The install/uninstall scripts are stdlib-only and cross-platform (no native component). Seeding is gated strictly on ``if args.app:`` (`backend/main.py:945`), placed after all CLI-return paths and before ``load_app_config()``.

2. **\*\*08 recorded honestly and matches code.\*\*** S7 row flipped to ? with PR #310; S8 08 "RESOLVED 2026-06-22" states the exact triple; S9 DoD checkbox flipped; changelog entry added. All consistent with ``write_light_default_config``.

3. **\*\*Tests assert the load-bearing behavior, not tautologies.\*\*** No-overwrite idempotency is proven by preserving a user's ``gemini`` choice (``test_seed_does_not_overwrite_existing``); the EFFECTIVE config is proven by running the seeded file through the loader and asserting ``skip_ai_handoff=True`` + ``storage.backend=local`` arrive from the ``truly-local`` PRESET (``test_seeded_config_loads_to_truly_local_motion``); the ``--app`` wiring is proven end-to-end in a subprocess.

4. **\*\*No real flakiness.\*\*** The subprocess seeding test isolates via ``RALLY_APP_HOME`?tmp + `cwd=tmp_path`` + free port + 90s deadline, and checks the file before polling process exit. The seeded ``truly-local`` config produces at most WARN-level startup checks (``backend/config/startup.py:187``), never an ERROR, so ``_enforce_startup_or_exit`` can't abort the test server.

5. **\*\*`--server`/CLI byte-identity preserved.\*\*** Seeding is the only new side effect and is ``--app``-only. 206 config/startup/server/app\_home tests pass unchanged.

Local verification: 22 new tests pass (``tests/test_p2b_install.py`` + ``tests/test_app_launcher.py``), 206 related tests pass, ``ruff check`` clean, ``mypy`` clean on changed files, installer ``--dry-run`` resolves the Windows app-home correctly, and the script ``default_os_app_home`` mirror matches the engine resolver exactly (``Khelsutra``, same precedence).

The two nits (no parity test between the duplicated ``default_os_app_home`` and the engine source of truth; install/uninstall IO orchestration covered only by ``--dry-run``) are optional follow-ups, consistent with the repo's pure-logic-locally / machine-side-to-owner convention.

Relevant files: ``C:\Users\avidu\Projects\bhi-p2b\backend\main.py``,  
``C:\Users\avidu\Projects\bhi-p2b\backend\config\loader.py``,  
``C:\Users\avidu\Projects\bhi-p2b\backend\config\presets.py``,  
``C:\Users\avidu\Projects\bhi-p2b\scripts\install.py``,  
``C:\Users\avidu\Projects\bhi-p2b\scripts\uninstall.py``,  
``C:\Users\avidu\Projects\bhi-p2b\tests\test_p2b_install.py``,  
``C:\Users\avidu\Projects\bhi-p2b\tests\test_app_launcher.py``,  
``C:\Users\avidu\Projects\bhi-p2b\docs\PACKAGING_PLAN.md``.